

Ход выполнения работы

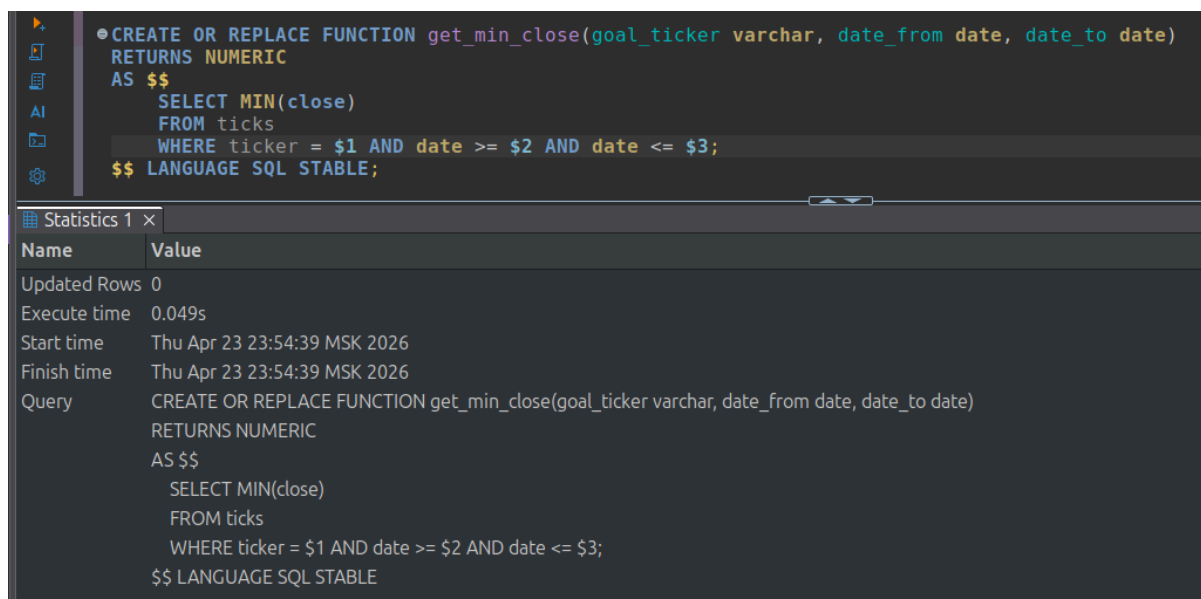
1. Сделать 2-3 функции с использованием SQL, возвращающих атомарные значения; можно попробовать использовать различные параметры «изменчивости». (Например: получить котировку по параметрам бумага и дата, минимальную цену за период).

Первой функцией, которую мы напишем и которая возвращает атомарное значение, будет функция получения минимальной цены закрытия (close) за указанный период - `get_min_close`. Для поиска минимальной цены закрытия за период нам необходимо, чтобы на вход были поданы следующие параметры: `goal_ticker` (типа данных VARCHAR) - краткое обозначение ценной бумаги, `date_from` (тип данных DATE) - с какой даты (включительно), `date_to` (тип данных DATE) - по какую дату (включительно). А возвращает эта функция минимальную цену закрытия (тип данных NUMERIC). Также стоит отметить, что был использован параметр изменчивости STABLE, так как без указания по умолчанию будет VOLATILE, что избыточно, ведь в функции происходит только чтение, обновлений не происходит, и в рамках одного запроса результат не изменится. Эта характеристика позволяет оптимизатору заменить множество вызовов этой функции одним. И после создания функции приведён пример её использования: находим минимальную цену закрытия бумаги SBER с 2026-01-05 по 2026-01-06.

```
CREATE OR REPLACE FUNCTION get_min_close(goal_ticker varchar, date_from
date, date_to date)
RETURNS NUMERIC
AS $$
    SELECT MIN(close)
    FROM ticks
    WHERE ticker = $1 AND date >= $2 AND date <= $3;
$$ LANGUAGE SQL STABLE;

SELECT get_min_close('SBER', '2026-01-05', '2026-01-06');
```

Создание этой функции представлено на скриншоте 1.1:

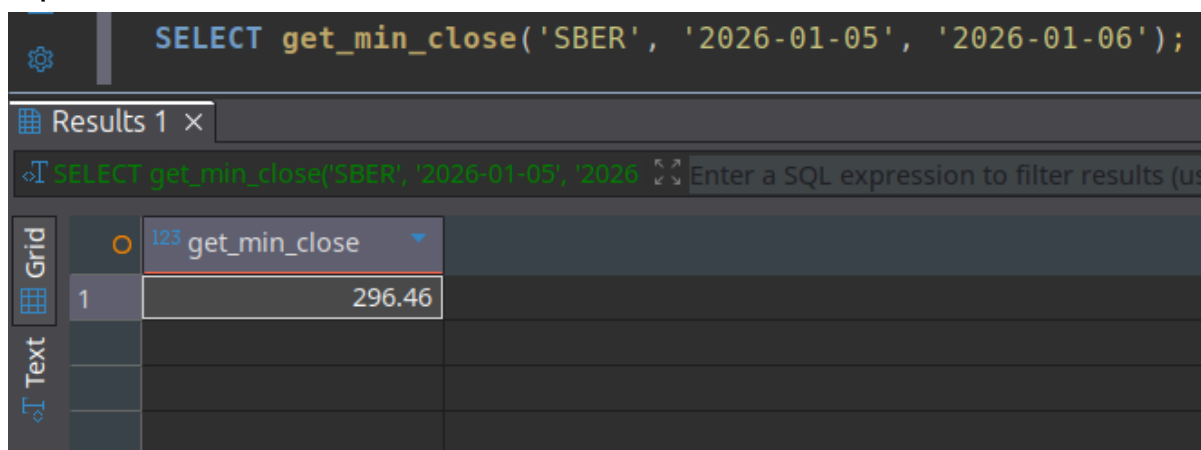


```
CREATE OR REPLACE FUNCTION get_min_close(goal_ticker varchar, date_from date, date_to date)
RETURNS NUMERIC
AS $$
    SELECT MIN(close)
    FROM ticks
    WHERE ticker = $1 AND date >= $2 AND date <= $3;
$$ LANGUAGE SQL STABLE;
```

Name	Value
Updated Rows	0
Execute time	0.049s
Start time	Thu Apr 23 23:54:39 MSK 2026
Finish time	Thu Apr 23 23:54:39 MSK 2026
Query	CREATE OR REPLACE FUNCTION get_min_close(goal_ticker varchar, date_from date, date_to date) RETURNS NUMERIC AS \$\$ SELECT MIN(close) FROM ticks WHERE ticker = \$1 AND date >= \$2 AND date <= \$3; \$\$ LANGUAGE SQL STABLE

Скриншот 1.1. Создание функции get_min_close

Пример работы функции get_min_close представлен на скриншоте 1.2:



```
SELECT get_min_close('SBER', '2026-01-05', '2026-01-06');
```

Grid	123 get_min_close
1	296.46

Скриншот 1.2. Пример работы функции get_min_close

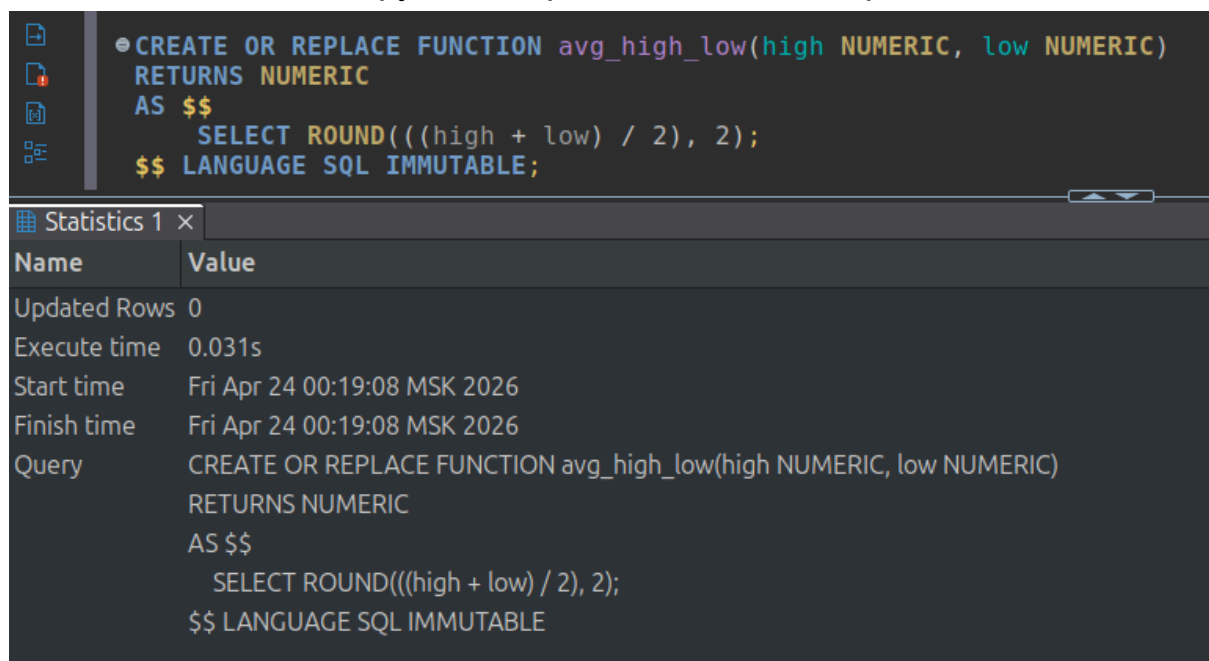
Второй функцией, которую мы напишем и которая возвращает атомарное значение, будет функция получения средней цены (по high и low). В функции необходимы входные параметры high (самая высокая цена) и low (самая низкая цена). А возвращать функция должна среднюю цену типа NUMERIC. В этой функции я установил параметр изменчивости IMMUTABLE, потому что результат зависит только от переданных чисел, это чистая математика, функция не делает ничего из того, что могло бы изменить результат при одинаковых входных данных. Эта характеристика позволяет оптимизатору предварительно вычислить функцию, когда она

вызывается в запросе с постоянными аргументами. И после создания функции приведён пример её использования.

```
CREATE OR REPLACE FUNCTION avg_high_low(high NUMERIC, low NUMERIC)
RETURNS NUMERIC
AS $$
    SELECT ROUND(((high + low) / 2), 2);
$$ LANGUAGE SQL IMMUTABLE;

SELECT ticker, date, high, low, avg_high_low(high, low) AS "avg_high_low"
FROM ticks
WHERE ticker = 'SBER';
```

Создание этой функции представлено на скриншоте 1.1:



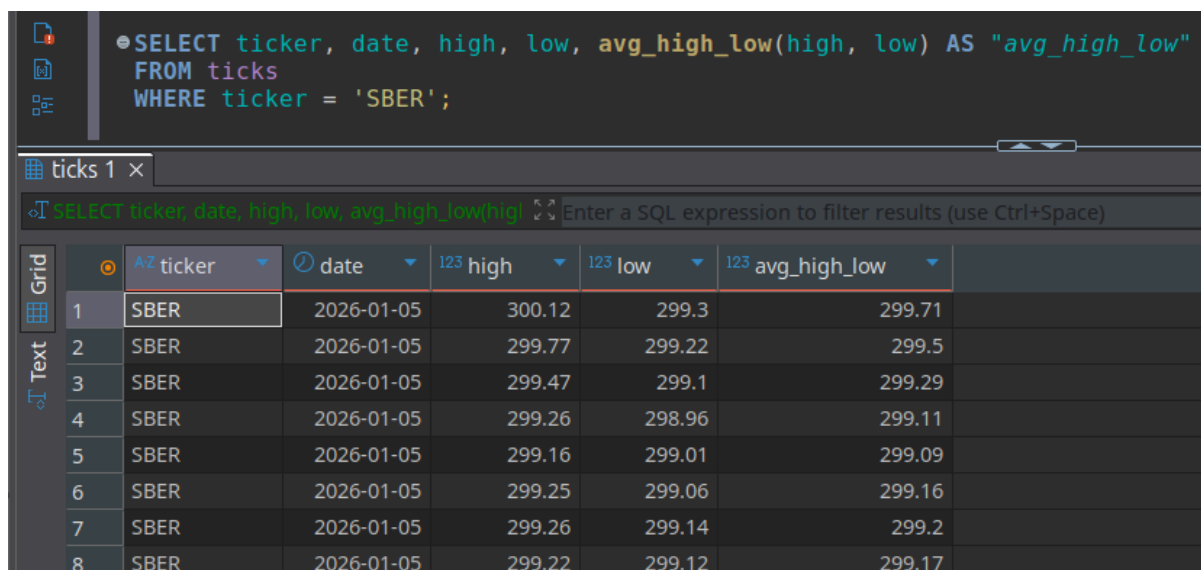
The screenshot shows a SQL IDE interface. The top pane displays the SQL code for creating the `avg_high_low` function. The bottom pane shows the execution statistics for the query.

```
CREATE OR REPLACE FUNCTION avg_high_low(high NUMERIC, low NUMERIC)
RETURNS NUMERIC
AS $$
    SELECT ROUND(((high + low) / 2), 2);
$$ LANGUAGE SQL IMMUTABLE;
```

Name	Value
Updated Rows	0
Execute time	0.031s
Start time	Fri Apr 24 00:19:08 MSK 2026
Finish time	Fri Apr 24 00:19:08 MSK 2026
Query	CREATE OR REPLACE FUNCTION avg_high_low(high NUMERIC, low NUMERIC) RETURNS NUMERIC AS \$\$ SELECT ROUND(((high + low) / 2), 2); \$\$ LANGUAGE SQL IMMUTABLE

Скриншот 1.3. Создание функции `avg_high_low`

Пример работы функции `avg_high_low` представлен на скриншоте 1.4:



```

SELECT ticker, date, high, low, avg_high_low(high, low) AS "avg_high_low"
FROM ticks
WHERE ticker = 'SBER';

```

	AZ ticker	date	high	low	avg_high_low
1	SBER	2026-01-05	300.12	299.3	299.71
2	SBER	2026-01-05	299.77	299.22	299.5
3	SBER	2026-01-05	299.47	299.1	299.29
4	SBER	2026-01-05	299.26	298.96	299.11
5	SBER	2026-01-05	299.16	299.01	299.09
6	SBER	2026-01-05	299.25	299.06	299.16
7	SBER	2026-01-05	299.26	299.14	299.2
8	SBER	2026-01-05	299.22	299.12	299.17

Скриншот 1.4. Пример работы функции avg_high_low

Третьей функцией, которую мы напишем и которая возвращает атомарное значение, будет функция удаления котировок, имеющих дату меньше заданной. Функция будет возвращать количество удаленных строк (значение типа INTEGER). Входным параметром будет дата - именно от нее зависит какие котировки будут удалены. С помощью CTE мы получим id всех удаленных строк и уже потом в SELECT-запросе выведем количество всех удаленных строк - это и будет значение, которое вернет функция. Эта функция будет с параметром изменчивости VOLATILE поскольку функция меняет базу данных. VOLATILE это значение по умолчанию, но для наглядности все равно укажем. И после создания этой функции проверим как она работает на примере.

```

CREATE OR REPLACE FUNCTION delete_old_ticks(
    delete_before_date DATE
)
RETURNS INTEGER
AS $$
    WITH deleted AS (
        DELETE FROM ticks
        WHERE date < delete_before_date
        RETURNING id
    )
    SELECT COUNT(*) FROM deleted;
$$ LANGUAGE SQL VOLATILE;

SELECT delete_old_ticks('2026-02-01');

```

Создание этой функции представлено на скриншоте 1.5:

The screenshot displays a database IDE interface. The top pane shows the SQL code for creating a function named `delete_old_ticks`. The function takes a `delete_before_date` of type `DATE` and returns an `INTEGER`. It uses a `WITH` clause to create a temporary table `deleted` by deleting rows from the `ticks` table where the `date` is less than `delete_before_date`. The function then returns the count of rows in the `deleted` table. The bottom pane shows the execution statistics for this query.

```
CREATE OR REPLACE FUNCTION delete_old_ticks(  
    delete_before_date DATE  
)  
    RETURNS INTEGER  
    AS $$  
        WITH deleted AS (  
            DELETE FROM ticks  
            WHERE date < delete_before_date  
            RETURNING id  
        )  
        SELECT COUNT(*) FROM deleted;  
    $$ LANGUAGE SQL VOLATILE;
```

Name	Value
Updated Rows	0
Execute time	0.036s
Start time	Fri Apr 24 00:32:54 MSK 2026
Finish time	Fri Apr 24 00:32:54 MSK 2026
Query	CREATE OR REPLACE FUNCTION delete_old_ticks(delete_before_date DATE) RETURNS INTEGER AS \$\$ WITH deleted AS (DELETE FROM ticks WHERE date < delete_before_date RETURNING id) SELECT COUNT(*) FROM deleted; \$\$ LANGUAGE SQL VOLATILE

Скриншот 1.5. Создание функции `delete_old_tricks`

Как выглядит вывод `SELECT * FROM ticks` перед выполнением примера показано на скриншоте 1.6:

The screenshot shows a database interface with two SQL queries in the top panel:

```
SELECT delete_old_ticks('2026-02-01');
SELECT * FROM ticks;
```

Below the queries, a table titled "ticks 1" is displayed. The table has 11 columns: id, ticker, per, date, time, open, high, low, close, and vol. The data represents stock ticks for SBER on 2026-01-05.

	id	ticker	per	date	time	open	high	low	close	vol
1	1	SBER	1	2026-01-05	07:00:00	299.99	300.12	299.3	299.77	91,164
2	2	SBER	1	2026-01-05	07:01:00	299.61	299.77	299.22	299.47	71,609
3	3	SBER	1	2026-01-05	07:02:00	299.47	299.47	299.1	299.16	43,542
4	4	SBER	1	2026-01-05	07:03:00	299.25	299.26	298.96	299.2	122,998
5	5	SBER	1	2026-01-05	07:04:00	299.02	299.16	299.01	299.08	32,275
6	6	SBER	1	2026-01-05	07:05:00	299.06	299.25	299.06	299.15	15,729
7	7	SBER	1	2026-01-05	07:06:00	299.21	299.26	299.14	299.14	8,897
8	8	SBER	1	2026-01-05	07:07:00	299.14	299.22	299.12	299.13	6,872
9	9	SBER	1	2026-01-05	07:08:00	299.13	299.13	298.87	298.9	27,270
10	10	SBER	1	2026-01-05	07:09:00	298.9	298.99	298.88	298.98	7,164
11	11	SBER	1	2026-01-05	07:10:00	298.98	299.01	298.96	298.99	14,815
12	12	SBER	1	2026-01-05	07:11:00	299	299.2	298.99	299.14	75,633
13	13	SBER	1	2026-01-05	07:12:00	299.2	299.22	298.77	298.98	51,319
14	14	SBER	1	2026-01-05	07:13:00	299.05	299.17	299.05	299.17	384

Скриншот 1.6. Значения таблицы ticks перед выполнением примера

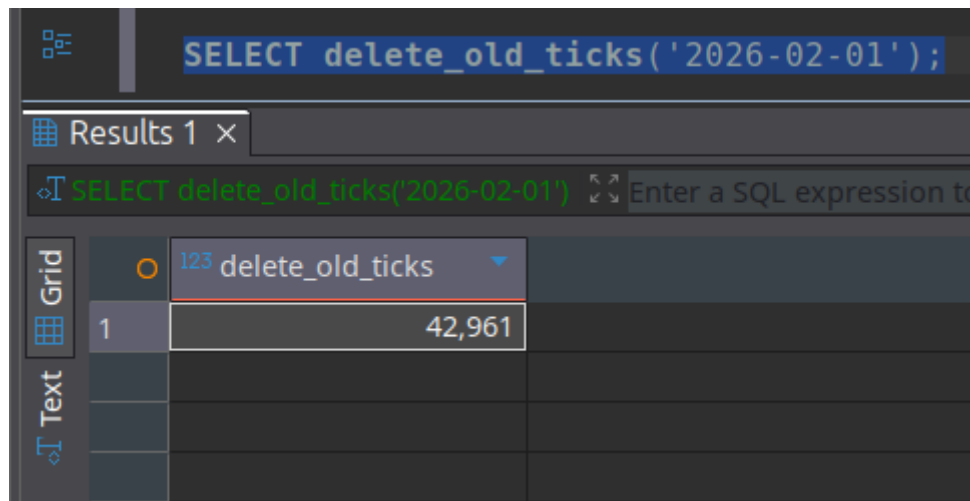
Как выглядит вывод SELECT * FROM ticks после выполнения примера показано на скриншоте 1.7:

The screenshot shows the same database interface, but the SQL query is now "SELECT * FROM ticks;". The table "ticks 1" displays the result of this query, showing ticks for SBER on 2026-02-01.

	id	ticker	per	date	time	open	high	low	close	vol
1	21,610	SBER	1	2026-02-01	09:59:00	305.72	305.72	305.72	305.72	728
2	21,611	SBER	1	2026-02-01	10:00:00	305.72	305.73	305.41	305.52	6,494
3	21,612	SBER	1	2026-02-01	10:01:00	305.62	305.72	305.52	305.71	2,865
4	21,613	SBER	1	2026-02-01	10:02:00	305.65	305.72	305.52	305.64	2,937
5	21,614	SBER	1	2026-02-01	10:03:00	305.64	305.66	305.53	305.57	436
6	21,615	SBER	1	2026-02-01	10:04:00	305.57	305.57	305.54	305.57	355
7	21,616	SBER	1	2026-02-01	10:05:00	305.57	305.68	305.56	305.68	393
8	21,617	SBER	1	2026-02-01	10:06:00	305.68	305.7	305.63	305.7	1,004
9	21,618	SBER	1	2026-02-01	10:07:00	305.69	305.71	305.65	305.7	1,117
10	21,619	SBER	1	2026-02-01	10:08:00	305.7	305.78	305.65	305.78	11,710
11	21,620	SBER	1	2026-02-01	10:09:00	305.78	305.81	305.76	305.81	750
12	21,621	SBER	1	2026-02-01	10:10:00	305.81	305.82	305.78	305.82	623
13	21,622	SBER	1	2026-02-01	10:11:00	305.82	305.82	305.78	305.79	291
14	21,623	SBER	1	2026-02-01	10:12:00	305.79	305.8	305.79	305.8	478
15	21,624	SBER	1	2026-02-01	10:13:00	305.8	305.8	305.78	305.8	1,777

Скриншот 1.7. Значения таблицы ticks после выполнения примера

Сам вызов функции приведён на скриншоте 1.8:



Скриншот 1.8. Пример вызова функции delete_old_ticks

2. Сделать функцию PL/pgSQL и примеры вызова в запросе.

Функция определяет дневной результат торгов по заданному тикеру: сравнивает первую цену открытия дня с последней ценой закрытия и возвращает таблицу из одной строки с тикером, ценами и флагом - Рост, Падение или Без изменений. Также были написаны примеры вызова в запросе:

```
CREATE OR REPLACE FUNCTION get_close_with_flag(
    goal_ticker VARCHAR,
    goal_date DATE
)
RETURNS TABLE(
    out_ticker VARCHAR,
    out_close NUMERIC,
    out_open NUMERIC,
    out_flag VARCHAR(20)
)
AS $$
DECLARE
    v_close NUMERIC;
    v_open NUMERIC;
BEGIN
    SELECT open INTO v_open
    FROM ticks
    WHERE ticker = goal_ticker
        AND date = goal_date
        ORDER BY time
    LIMIT 1;
    SELECT close INTO v_close
    FROM ticks
    WHERE ticker = goal_ticker
        AND date = goal_date
        ORDER BY time DESC
    LIMIT 1;
```

```

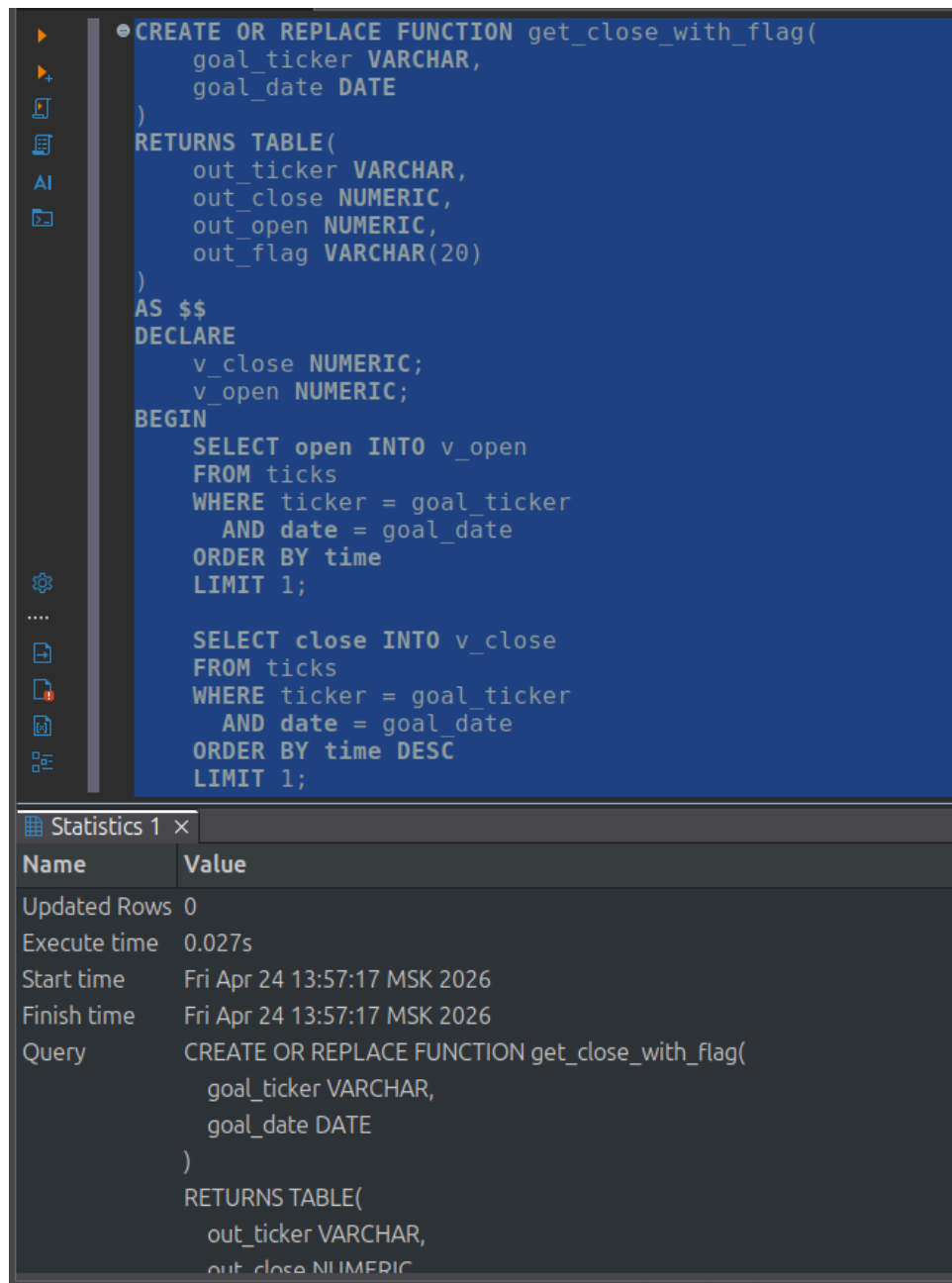
    IF v_close IS NULL THEN
        RAISE EXCEPTION 'Нет данных по тикеру % на дату %', goal_ticker,
goal_date;
    END IF;
    IF v_close > v_open THEN
        RETURN QUERY SELECT goal_ticker, v_close, v_open, 'Рост'::VARCHAR;
    ELSIF v_close < v_open THEN
        RETURN QUERY SELECT goal_ticker, v_close, v_open, 'Падение'::VARCHAR;
    ELSE
        RETURN QUERY SELECT goal_ticker, v_close, v_open, 'Без
изменений'::VARCHAR;
    END IF;
END;
$$ LANGUAGE plpgsql STABLE;

SELECT * FROM get_close_with_flag('SBER', '2026-04-01');

SELECT *
FROM (
    SELECT * FROM get_close_with_flag('SBER', '2026-02-01')
    UNION ALL
    SELECT * FROM get_close_with_flag('SBER', '2026-02-02')
    UNION ALL
    SELECT * FROM get_close_with_flag('SBER', '2026-02-03')
) AS results
WHERE out_flag = 'Падение';

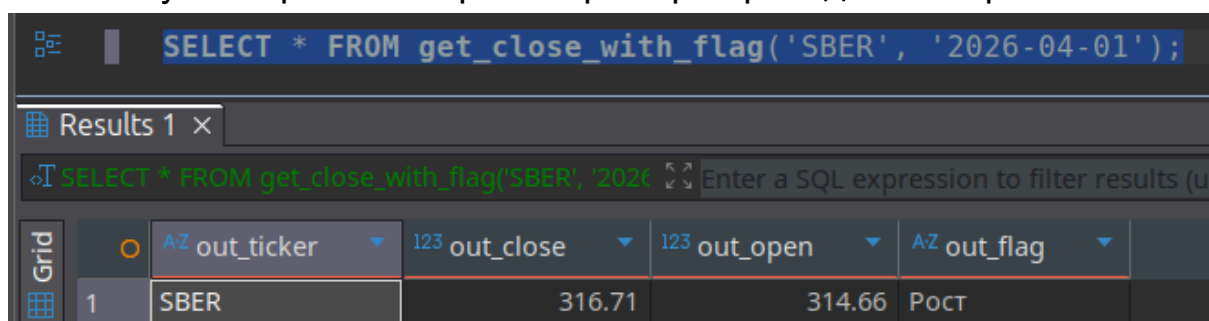
```

Создание этой функции представлено на скриншоте 2.1:



Скриншот 2.1. Создание функции get_close_with_flag

Результат работы первого примера приведён на скриншоте 2.2:



Скриншот 2.2. Результат работы первого примера

Результат работы второго примера приведён на скриншоте 2.3:

The screenshot shows a SQL IDE with a query editor and a results pane. The query in the editor is:

```
SELECT *
FROM (
    SELECT * FROM get_close_with_flag('SBER', '2026-02-01')
    UNION ALL
    SELECT * FROM get_close_with_flag('SBER', '2026-02-02')
    UNION ALL
    SELECT * FROM get_close_with_flag('SBER', '2026-02-03')
) AS results
WHERE out_flag = 'Падение';
```

The results pane shows a table with 5 columns: out_ticker, out_close, out_open, out_flag, and an empty column. The data is as follows:

	out_ticker	out_close	out_open	out_flag	
1	SBER	304.16	306.48	Падение	
2	SBER	303.81	304.56	Падение	

Скриншот 2.3. Результат работы второго примера

Также продемонстрирую как сработает RAISE EXCEPTION на скриншоте 2.4:

The screenshot shows a SQL IDE with a query editor and a results pane. The query in the editor is:

```
SELECT * FROM get_close_with_flag('SBER', '2026-05-01');
```

The results pane shows an error message:

```
SQL Error [P0001]: ERROR: Нет данных по тикеру SBER на дату 2026-05-01
Where: PL/pgSQL function get_close_with_flag(character varying,date) line 21 at RAISE
Error position:
```

Скриншот 2.3. Результат работы третьего примера

3. Написать функцию, которая возвращает таблицу, и примеры вызова в запросе.

Ниже приведён скрипт создания функции, возвращающей таблицу - статистику всех ценных бумаг в исходной таблице ticks по указанному периоду. После создания функции приведены примеры вызова в двух запросах. Первый пример - использование вызова

функции как обычную таблицу в FROM. Второй пример - использование вызова функции как таблицу в операции JOIN.

```
CREATE OR REPLACE FUNCTION get_tickers_stats(date_from date, date_to date)
RETURNS TABLE(ticker VARCHAR, min_price NUMERIC, max_price NUMERIC,
avg_price NUMERIC, total_vol BIGINT, days_count INTEGER)
AS $$
    SELECT
        ticker,
        MIN(low),
        MAX(high),
        ROUND(AVG(close), 2),
        SUM(vol),
        COUNT(DISTINCT date)
    FROM ticks
    WHERE date BETWEEN date_from AND date_to
    GROUP BY ticker;
$$ LANGUAGE SQL STABLE;

SELECT ticker, total_vol, avg_price
FROM get_tickers_stats('2026-01-05', '2026-04-29')
ORDER BY total_vol DESC;

SELECT ticker_stats.ticker, total_vol / vol AS volume_ratio
FROM get_tickers_stats('2026-01-05', '2026-04-29') ticker_stats
JOIN ticks ON ticker_stats.ticker = ticks.ticker;
```

Создание этой функции представлено на скриншоте 3.1:

The screenshot displays a database IDE with a SQL editor and a 'Statistics 1' window. The SQL editor contains the following code:

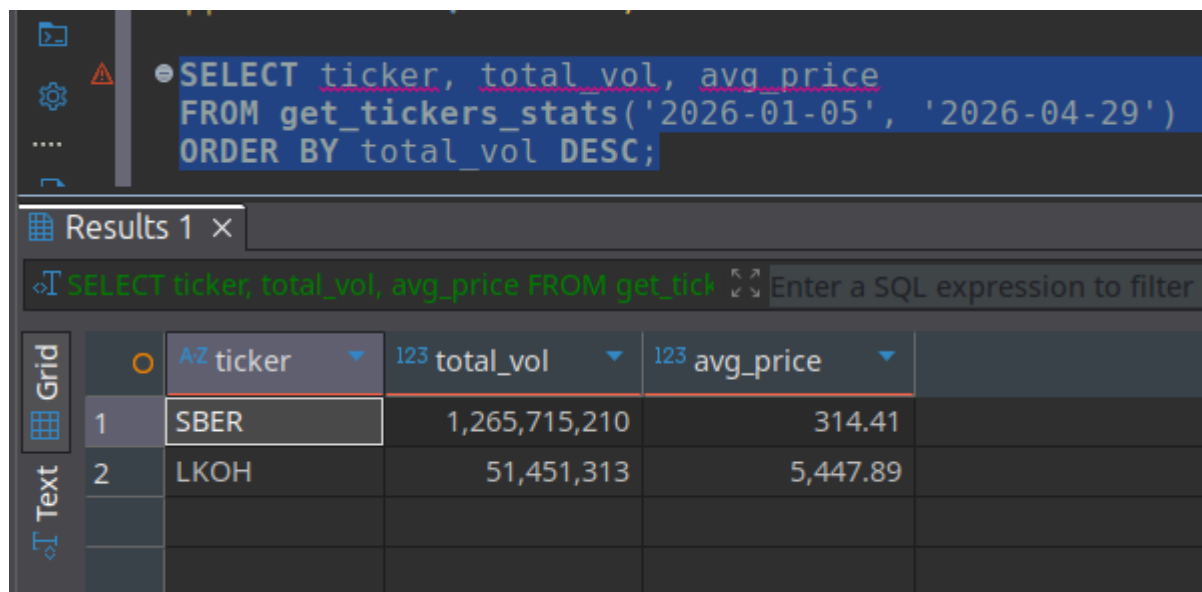
```
CREATE OR REPLACE FUNCTION get_tickers_stats(date_from date, date_to date)
RETURNS TABLE(ticker VARCHAR, min_price NUMERIC, max_price NUMERIC, avg_price NUMERIC, total_vol BIGINT, days_count INTEGER)
AS $$
    SELECT
        ticker,
        MIN(low),
        MAX(high),
        ROUND(AVG(close), 2),
        SUM(vol),
        COUNT(DISTINCT date)
    FROM ticks
    WHERE date BETWEEN date_from AND date_to
    GROUP BY ticker;
$$ LANGUAGE SQL STABLE;
```

The 'Statistics 1' window shows the following details:

Name	Value
Updated Rows	0
Execute time	0.068s
Start time	Fri Apr 24 12:01:00 MSK 2026
Finish time	Fri Apr 24 12:01:00 MSK 2026
Query	CREATE OR REPLACE FUNCTION get_tickers_stats(date_from date, date_to date) RETURNS TABLE(ticker VARCHAR, min_price NUMERIC, max_price NUMERIC, avg_price NUMERIC, total_vol BIGINT, days_count INTEGER) AS \$\$ SELECT ticker, MIN(low), MAX(high), ROUND(AVG(close), 2), SUM(vol), COUNT(DISTINCT date) FROM ticks WHERE date BETWEEN date_from AND date_to GROUP BY ticker; \$\$ LANGUAGE SQL STABLE

Скриншот 3.1. Создание функции get_tickers_stats

Результат первого примера использования представлен на скриншоте 3.2:

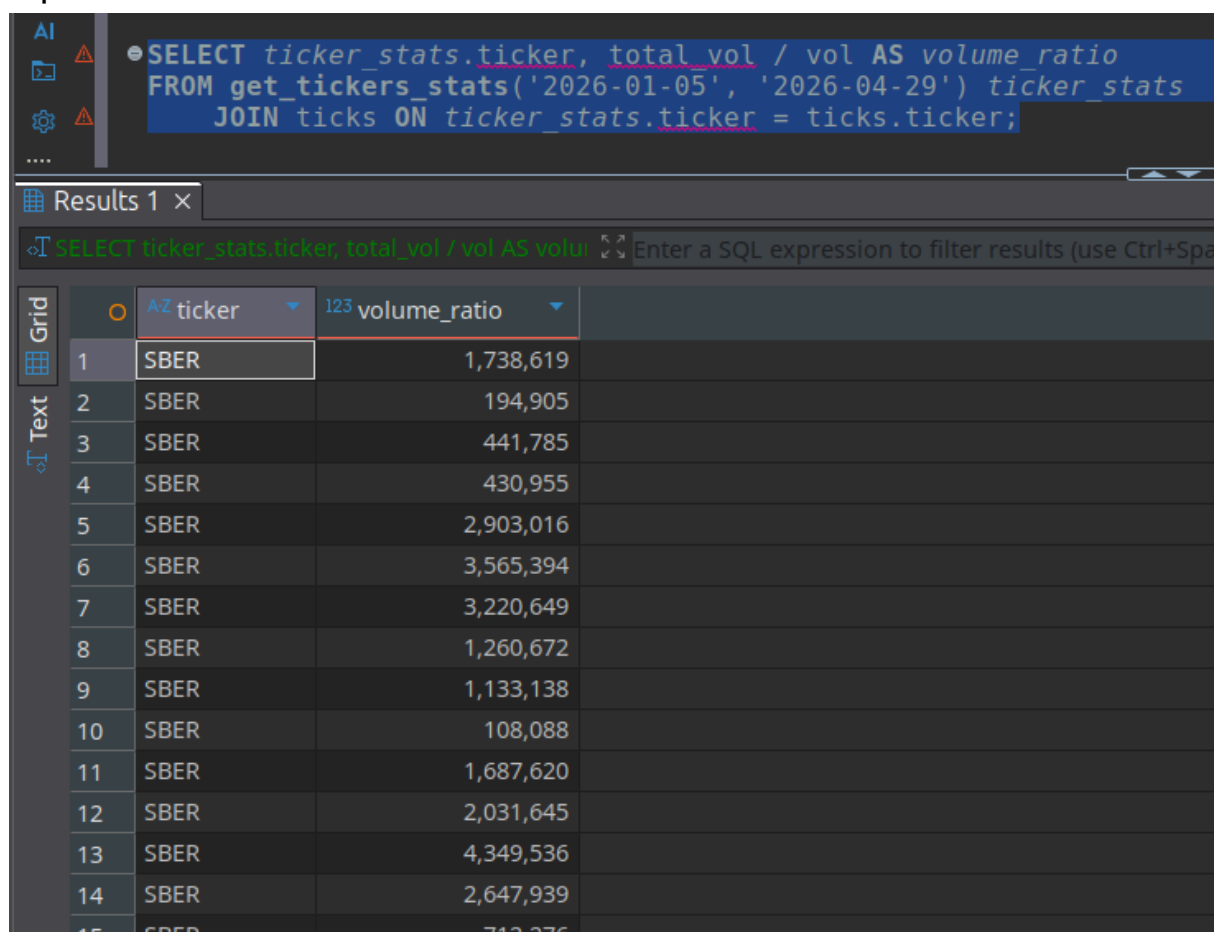


The screenshot shows a SQL IDE interface. At the top, a query is entered in a text editor: `SELECT ticker, total_vol, avg_price FROM get_tickers_stats('2026-01-05', '2026-04-29') ORDER BY total_vol DESC;`. Below the editor, the 'Results' tab is active, displaying a table with 3 columns: 'ticker', 'total_vol', and 'avg_price'. The table is sorted by 'total_vol' in descending order. The first two rows are visible: 'SBER' with a volume of 1,265,715,210 and an average price of 314.41, and 'LKOH' with a volume of 51,451,313 and an average price of 5,447.89.

	ticker	total_vol	avg_price
1	SBER	1,265,715,210	314.41
2	LKOH	51,451,313	5,447.89

Скриншот 3.2. Первый пример использования

Результат второго примера использования представлен на скриншоте 3.3:



The screenshot shows a SQL IDE interface. At the top, a query is entered in a text editor: `SELECT ticker_stats.ticker, total_vol / vol AS volume_ratio FROM get_tickers_stats('2026-01-05', '2026-04-29') ticker_stats JOIN ticks ON ticker_stats.ticker = ticks.ticker;`. Below the editor, the 'Results' tab is active, displaying a table with 3 columns: 'ticker', 'total_vol / vol AS volume_ratio', and an empty column. The table is sorted by 'volume_ratio' in descending order. The first 15 rows are visible, all with 'SBER' as the ticker. The volume ratios range from 1,738,619 down to 712,276.

	ticker	total_vol / vol AS volume_ratio
1	SBER	1,738,619
2	SBER	194,905
3	SBER	441,785
4	SBER	430,955
5	SBER	2,903,016
6	SBER	3,565,394
7	SBER	3,220,649
8	SBER	1,260,672
9	SBER	1,133,138
10	SBER	108,088
11	SBER	1,687,620
12	SBER	2,031,645
13	SBER	4,349,536
14	SBER	2,647,939
15	SBER	712,276

Скриншот 3.3. Второй пример использования

4. Написать функцию, которая возвращает SETOF, и примеры вызова в запросе.

Была написана функция, которая возвращает SETOF RECORD, а именно строки, где vol превышает средний объем по тикеру в n или более раз. Также были написаны запросы для демонстрации работы:

```
CREATE OR REPLACE FUNCTION get_high_volume_ticks(n INTEGER)
RETURNS SETOF RECORD
AS $$
    WITH tickers_avg_vol as (SELECT ticker, avg(vol) as avg_vol FROM ticks
GROUP BY ticker)
    SELECT t.ticker as ticker, date, time, t.vol as ticker_vol,
    tav.avg_vol as ticker_avg_vol
    FROM ticks t join tickers_avg_vol tav on t.ticker = tav.ticker
    WHERE ROUND(t.vol / tav.avg_vol) > n
$$ LANGUAGE SQL STABLE;

SELECT get_high_volume_ticks(2);

SELECT *
```

Создание этой функции представлено на скриншоте 4.1:

The screenshot shows a database IDE with a query editor and a statistics panel. The query editor contains the following SQL code:

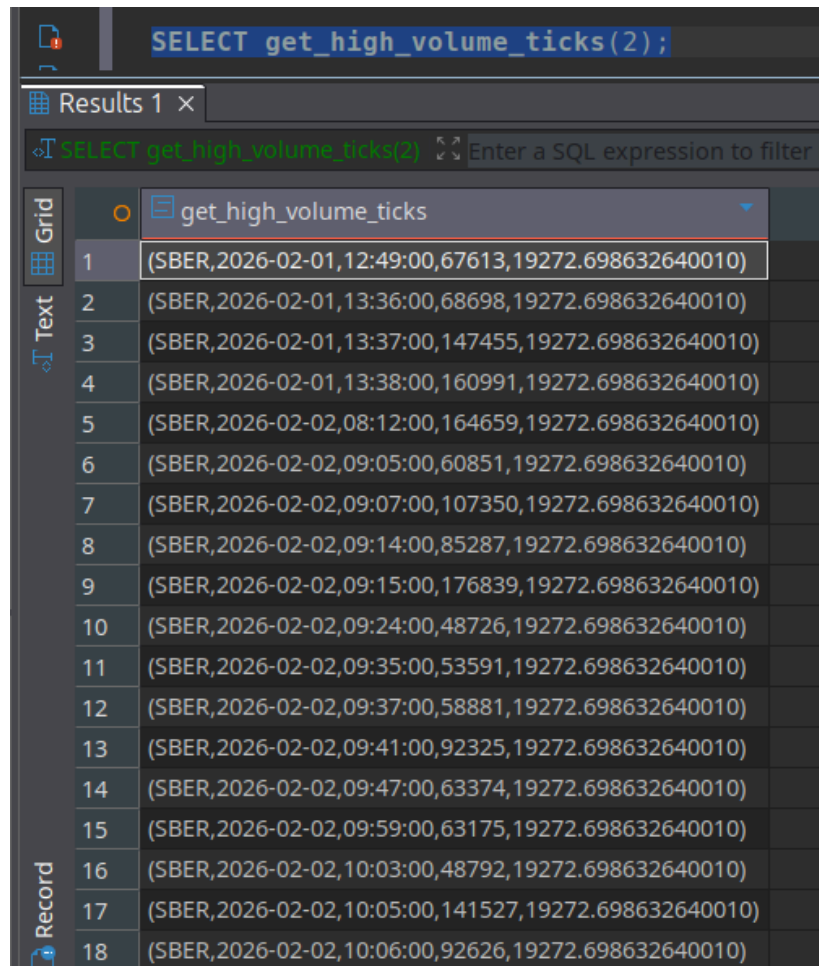
```
CREATE OR REPLACE FUNCTION get_high_volume_ticks(n INTEGER)
RETURNS SETOF RECORD
AS $$
    WITH tickers_avg_vol as (SELECT ticker, avg(vol) as avg_vol FROM ticks GROUP BY ticker)
    SELECT t.ticker as ticker, date, time, t.vol as ticker_vol, tav.avg_vol as ticker_avg_vol
    FROM ticks t join tickers_avg_vol tav on t.ticker = tav.ticker
    WHERE ROUND(t.vol / tav.avg_vol) > n
$$ LANGUAGE SQL STABLE;
```

The statistics panel shows the following information:

Name	Value
Updated Rows	0
Execute time	0.059s
Start time	Fri Apr 24 12:41:11 MSK 2026
Finish time	Fri Apr 24 12:41:11 MSK 2026
Query	CREATE OR REPLACE FUNCTION get_high_volume_ticks(n INTEGER) RETURNS SETOF RECORD AS \$\$ WITH tickers_avg_vol as (SELECT ticker, avg(vol) as avg_vol FROM ticks GROUP BY ticker) SELECT t.ticker as ticker, date, time, t.vol as ticker_vol, tav.avg_vol as ticker_avg_vol FROM ticks t join tickers_avg_vol tav on t.ticker = tav.ticker WHERE ROUND(t.vol / tav.avg_vol) > n \$\$ LANGUAGE SQL STABLE

Скриншот 4.1. Создание функции

Пример 1 представлен на скриншоте 4.2. Этот запрос возвращает одну колонку с кортежами. PostgreSQL не знает, как разбить кортеж на колонки, потому что в RETURNS SETOF RECORD структура не задана. Поэтому он показывает весь кортеж как одно значение в одной колонке.



The screenshot shows a PostgreSQL query window with the query `SELECT get_high_volume_ticks(2);` executed. The results are displayed in a table with 18 rows. The table has a single column named `get_high_volume_ticks`. Each row contains a tuple of values: `(SBER, 2026-02-01, 12:49:00, 67613, 19272.698632640010)`.

	get_high_volume_ticks
1	(SBER,2026-02-01,12:49:00,67613,19272.698632640010)
2	(SBER,2026-02-01,13:36:00,68698,19272.698632640010)
3	(SBER,2026-02-01,13:37:00,147455,19272.698632640010)
4	(SBER,2026-02-01,13:38:00,160991,19272.698632640010)
5	(SBER,2026-02-02,08:12:00,164659,19272.698632640010)
6	(SBER,2026-02-02,09:05:00,60851,19272.698632640010)
7	(SBER,2026-02-02,09:07:00,107350,19272.698632640010)
8	(SBER,2026-02-02,09:14:00,85287,19272.698632640010)
9	(SBER,2026-02-02,09:15:00,176839,19272.698632640010)
10	(SBER,2026-02-02,09:24:00,48726,19272.698632640010)
11	(SBER,2026-02-02,09:35:00,53591,19272.698632640010)
12	(SBER,2026-02-02,09:37:00,58881,19272.698632640010)
13	(SBER,2026-02-02,09:41:00,92325,19272.698632640010)
14	(SBER,2026-02-02,09:47:00,63374,19272.698632640010)
15	(SBER,2026-02-02,09:59:00,63175,19272.698632640010)
16	(SBER,2026-02-02,10:03:00,48792,19272.698632640010)
17	(SBER,2026-02-02,10:05:00,141527,19272.698632640010)
18	(SBER,2026-02-02,10:06:00,92626,19272.698632640010)

Скриншот 4.2. Пример 1

Пример 2 представлен на скриншоте 4.3: когда мы добавляем `AS f(колонка1 тип, колонка2 тип, ...)`, PostgreSQL понимает структуру и разбирает каждый кортеж на отдельные колонки:

SELECT * FROM get_high_volume_ticks(2) AS f(ticker VARCHAR, date DATE, time TIME, ticker_vol INTEGER, ticker_avg_vol NUMERIC);						
Results 1 x						
SELECT * FROM get_high_volume_ticks(2) AS f; Enter a SQL expression to filter results (use Ctrl+Space)						
	ticker	date	time	ticker_vol	ticker_avg_vol	
1	SBER	2026-02-01	12:49:00	67,613	19,272.69863264	
2	SBER	2026-02-01	13:36:00	68,698	19,272.69863264	
3	SBER	2026-02-01	13:37:00	147,455	19,272.69863264	
4	SBER	2026-02-01	13:38:00	160,991	19,272.69863264	
5	SBER	2026-02-02	08:12:00	164,659	19,272.69863264	
6	SBER	2026-02-02	09:05:00	60,851	19,272.69863264	
7	SBER	2026-02-02	09:07:00	107,350	19,272.69863264	
8	SBER	2026-02-02	09:14:00	85,287	19,272.69863264	
9	SBER	2026-02-02	09:15:00	176,839	19,272.69863264	
10	SBER	2026-02-02	09:24:00	48,726	19,272.69863264	
11	SBER	2026-02-02	09:35:00	53,591	19,272.69863264	
12	SBER	2026-02-02	09:37:00	58,881	19,272.69863264	
13	SBER	2026-02-02	09:41:00	92,325	19,272.69863264	
14	SBER	2026-02-02	09:47:00	63,374	19,272.69863264	
15	SBER	2026-02-02	09:59:00	63,175	19,272.69863264	
16	SBER	2026-02-02	10:03:00	48,792	19,272.69863264	
17	SBER	2026-02-02	10:05:00	141,527	19,272.69863264	
18	SBER	2026-02-02	10:06:00	92,626	19,272.69863264	

Скриншот 4.3. Пример 2